

AI CYBER ATTACK PREDICTION ENGINE

System Analysis, Graph Intelligence Architecture & Complete Operational Tutorial

Domain: Graph Machine Learning & Proactive Cybersecurity | **Framework:** PyTorch Geometric, XGBoost, FastAPI, React
Institution: VIT — Department of Computer Engineering | **Date:** August 2026

1. Executive Summary & Problem Formulation

Traditional enterprise defense solutions (Intrusion Detection Systems, Firewalls, SIEMs) operate reactively — they detect threats only after execution begins, answering *'what happened?'* rather than *'what will propagate next?'* In modern complex enterprise and campus network environments, security teams are overloaded by alert volumes without actionable prioritization.

The **AI Cyber Attack Prediction Engine** transforms network defense into a **temporal, graph-aware, risk-prioritized prediction engine**. By fusing network flow features, graph topology centrality, temporal interaction order, and asset vulnerability into a cohesive dynamic risk score, it proactively alerts defenders to the next likely attack targets before compromise occurs.

2. System Architecture & Pipeline Layers

Pipeline Stage	Component & Technology	Function & Deliverable
1. Data Ingestion	CICIDS2017 + Campus Topology JSON	Ingests 80+ network traffic features across VLAN segments.
2. Preprocessing	Pandas, Cleaner, Time-Windowing	5-minute non-overlapping temporal windows, zero-leakage labels.
3. Graph Extraction	NetworkX, Topology Centrality	Extracts Degree, Betweenness, Closeness, and PageRank per window.
4. Multi-Model ML	XGBoost, GNN (GraphSAGE), LSTM	Predicts next-target attack probabilities per device.
5. Risk Engine	Dynamic 6-Factor Formulator	Combines Attack Probability, Anomaly, Vuln, Exposure, Criticality.
6. Explainable AI	SHAP (TreeExplainer)	Quantifies directional feature contributions per prediction.
7. Recommendation	Rule-Based Action Engine	Generates 10 prioritized actions across 7 defense categories.
8. Serving Layer	FastAPI Backend (Port 8000)	11 RESTful endpoints serving real-time predictions & topology.
9. Frontend UI	React, Vite, Cytoscape.js, Recharts	Cyber-themed interactive dashboard with graph & telemetry.

3. Empirical Evaluation & Model Benchmarks

All models were evaluated under strict chronological temporal splitting without lookahead data leakage. The table below details benchmark metrics across the 6 model configurations tested on the Wednesday attack dataset:

Model Architecture	Top-1	Top-3	Top-5	MRR	PR-AUC	F1 Score
Heuristic Baseline	1.00	1.00	1.00	1.00	0.26	0.40
XGBoost Baseline	1.00	1.00	1.00	1.00	1.00	0.97
Isolation Forest + XGBoost	1.00	1.00	1.00	1.00	1.00	0.97
Dynamic Risk Engine	1.00	1.00	1.00	1.00	1.00	1.00
GNN (GraphSAGE)	0.86	0.93	0.97	0.90	0.92	0.94
Temporal LSTM	0.57	0.86	1.00	0.71	0.59	0.92

4. Dynamic Risk & Explainability Formulation

Dynamic Risk Score Formula:

$$\text{DynamicRisk}(d) = (1/6) \cdot P_{\text{attack}}(d) + (1/6) \cdot S_{\text{anomaly}}(d) + (1/6) \cdot V_{\text{vuln}}(d) + (1/6) \cdot E_{\text{topo}}(d) + (1/6) \cdot C_{\text{asset}}(d) + (1/6) \cdot R_{\text{recency}}(d)$$

This multi-factor formulation ensures high-criticality assets (e.g. Database, Web Servers) receive immediate protection priority even when low-criticality workstations experience transient background probes. Feature attribution via SHAP TreeExplainer provides exact directional reasons (e.g. forward packet volume, VLAN exposure) for every alert.

5. Operational Tutorial: How to Start and Run the Project

Prerequisites

- **Python 3.10+** (with virtual environment configured at venv/)
- **Node.js 18+ & npm** (for the Vite + React frontend dashboard)
- **Windows / Linux / macOS** operating system

Step 1: Environment Verification

Open terminal in project root and verify the dataset and pre-trained models:

```
cd "c:\EDI\Sem 3\antitry1"
.\venv\Scripts\python.exe verify_all.py
```

Step 2: Start the FastAPI Backend Server

Launch the backend API service on port 8000:

```
.\venv\Scripts\python.exe -m uvicorn backend.app.main:app --port 8000 --reload
```

- Health Check: <http://127.0.0.1:8000/health>
- Interactive Swagger Docs: <http://127.0.0.1:8000/docs>

Step 3: Start the React Cyber Dashboard

In a second terminal window, start the Vite development server:

```
cd "c:\EDI\Sem 3\antitry1\frontend"
npm run dev
```

- Access Dashboard at: <http://127.0.0.1:5173>

Step 4: Using the Dashboard Features

1. **Timeline Window Bar:** Select time windows (e.g. W84–W97) to view temporal attack evolution.
2. **Interactive Network Topology:** Click any device node to inspect its risk profile and connections.
3. **Trigger On-Demand Scans:** Select model (XGBoost, GNN, Temporal) and click '■ Run AI Prediction Scan'.
4. **SHAP Waterfall Panel:** Inspect exact feature contributions driving risk scores.
5. **Attack Path Tracer:** View multi-hop attack propagation sequences with animated graph highlights.
6. **Defensive Action Playbook:** Review prioritized remediation steps (Isolation, SOC escalation, Port restrictions).

6. API Endpoints Reference

Endpoint	Method	Description & Parameters
/health	GET	Server health, loaded model flags, device & window counts.
/api/network	GET	Topology nodes & edges with optional ?window_id= overlay.
/api/risk	GET	Ranked 21-device dynamic risk scores and contributing factors.
/api/predictions	GET	Top-K predicted future attack targets (?top_k=5&model=xgboost).
/api/timeline	GET	All available time windows with attack event indicators.
/api/evaluation	GET	Benchmark comparison metrics for all 6 evaluated models.
/api/explanation/{id}	GET	Local SHAP feature waterfall breakdown for target device.
/api/recommendations/{id}	GET	Prioritized defensive actions and remediation rationales.
/api/attack-path/{id}	GET	Multi-hop attack propagation path from breach source.
/api/analyze	POST	Triggers real-time ML inference for selected window & model.